

Erick Epley

July 28th, 2026

IT FDN 110 GP Su 26

Assignment 05

[Github URL](#) (external site)

Advanced Collections and Error Handling

Introduction

This week I built upon my coding skills from last week's assignment, creating a program to read and write data from a user's input to a JSON file. This week was also our first time using Github, so after writing the program I learned how to upload the files as a new repository to the site for better peer review and communication about this program.

Writing the Program

To begin this assignment I started with the provided Starter File, which had a jumping off point featuring code from the previous week's assignment. Before defining the data constants and variables, I used a new line of code learned this week with the import function to import the relevant json information for the program. (Figure 1).



```
# Import the json
import json
```

Figure 1. Importing the JSON Information

With the import function added, I followed by defining the data constants and variables. (Figures 2 and 3).

```
# Define the Data Constants
MENU: str = ""
---- Course Registration Program ----
    Select from the following menu:
        1. Register a Student for a Course.
        2. Show current data.
        3. Save data to a file.
        4. Exit the program.
-----
"""
FILE_NAME: str = "Enrollments.json"
```

Figure 2. Data Constants

```
# Define the Data Variables
student_first_name: str = "" # Holds the first name of a student entered by the user.
student_last_name: str = "" # Holds the last name of a student entered by the user.
course_name: str = "" # Holds the name of a course entered by the user.
student_data: dict = {} # one row of student data
students: list = [] # a table of student data
file = None # Holds a reference to an opened file.
menu_choice: str = "" # Hold the choice made by the user.
```

Figure 3. Data Variables

This program was the first time using dictionaries. Last week we had defined the “student_data” dictionary as a list using the brackets, but this week I initialized the student_data variable as a dictionary using the curly brackets. The students list would remain a list for this project, except it would be a list of dictionaries. (Figure 3).

The next step was to open the “Enrollments.JSON” file which the program would be reading from and writing data to. At the beginning of the program, according to the acceptance criteria, the program would need to open and read from a JSON file. Since we were also working with exceptions this week, I used a try/except function to look in the file folder and make sure that there was already a file created. If there wasn’t a file, the program would explain that a file must exist before the program can run. (Figure 4).

At the beginning of the try/except function I used the open() command in read mode to open the JSON file with the name declared in the FILE_NAME data constant. I used the json.load() command to copy all of the current data from the file into the “students” list to be added to during this program’s run time. (Figure 4).

```

# Attempt to read the file, provide error handling for exceptions
try:
    # When the program starts, read the file data into a list of lists (table)
    # Extract the data from the file
    file = open(FILE_NAME, "r")
    students = json.load(file)
    file.close()

# FileNotFoundError exception
except FileNotFoundError as e:
    print(f"A file named: {FILE_NAME} must exist before running the script.")
    print("-- Technical Error Message --")
    print(e, e.__doc__, type(e), sep='\n')

# General Exception
except Exception as e:
    print("There was a non-specific error!")
    print("-- Technical Error Message --")
    print(e, e.__doc__, type(e), sep='\n')

# Finally check to see if the file has been closed
finally:
    # Check if a file object exists and is still open
    if file is not None and file.closed == False:
        file.close()

```

Figure 4. Reading the data from a JSON file with exceptions in case a file doesn't exist

Next I checked the while loop provided in the starter file. This loop was set to True and was currently coded to be used with a CSV file. Since I was working with JSON files today, I would need to update some of these categories to work with JSON files and dictionaries instead of CSV files and lists. (Figure 5).

For the first menu choice, the program would ask the user to input their first name, last name, and course name, and fill the `student_first_name`, `student_last_name`, and `course_name` variables respectively. The acceptance criteria for this program also asked that we had error management to catch if the user input invalid characters for the students first and last name, so inside of the if statement I added a try statement. (Figure 5).

I used the `input()` command to take in the user input, and I used the `.isalpha()` command to check the `student_first_name` or `student_last_name` variables after the user's input to check if the characters were all alphabetical. If there were any numbers, this would raise a Value Error to the user, which I later use an except statement to print out a relevant message telling the user to only use alphabetical characters for the first and last name inputs. I also added another exception which would be a general exception telling the user that an error occurred. (Figure 5).

```

# Input user data
if menu_choice == "1": # This will not work if it is an integer!
    # Begin Try/Except to ensure Names have the correct characters
    try:
        student_first_name = input("Enter the student's first name: ")
        if not student_first_name.isalpha():
            raise ValueError("The First Name Should Only Contain Letters.")
        student_last_name = input("Enter the student's last name: ")
        if not student_last_name.isalpha():
            raise ValueError("The Last Name Should Only Contain Letters.")
        course_name = input("Please enter the name of the course: ")
        student_data = {"FirstName": student_first_name,
                        "LastName": student_last_name,
                        "CourseName": course_name} # Format Data into a Dictionary
        students.append(student_data)
        print(f"You have registered {student_first_name} {student_last_name} for {course_name}.")

    # Value Error exception
    except ValueError as e:
        print(e) # Print the custom message
        print("-- Technical Error Message --")
        print(e.__doc__)

    # General exception
    except Exception as e:
        print("There was a non-specific error!")
        print("-- Technical Error Message --")
        print(e, e.__doc__, type(e), sep='\n')

```

Figure 5. Menu Choice 1 with an if loop and try statement

For the second menu choice I used an elif statement to look for the user to input “2”. For this input I needed to print out the data within the “students” list of “student_data” dictionaries in a comma separated format, so I used an f string and curly brackets within a for loop to look for student in students. For each student I indexed the student[“FirstName”], student[“LastName”], and student[“CourseName”] to go into each row of the “students” list and find the correct string value from the “student_data” dictionary. (Figure 6).

```

# Present the current data
elif menu_choice == "2":
    # Process the data to create and display a custom message
    print("-"*50)
    print("The Current Data is: ")
    for student in students:
        print(f"{student['FirstName']},{student['LastName']},{student['CourseName']}")
    print("-"*50)
    continue

```

Figure 6. Menu Choice 2 using a for loop

For the third menu choice I needed to save the current data, the data read from the file and the appended data from the user, to the JSON file.

I opened the file in write mode, then used the `json.dump()` command to write the current data to the file. Inside the `json.dump()` command I told it to write the “students” list to the “file” variable and I added an indent of 2 for organizational purposes. (Figure 7).

This menu choice would also need some error management to check if the file was the correct type, so I used a try statement which the file writing code would be nested into. I used an except statement to check for a Type Error, and if it was it would tell the user that the file needed to be in a JSON format. I also added a general exception for a non-specific error. (Figure 7).

Lastly I added a finally clause that would check to see if the file was closed. If it had not been closed already then this section would close the file using the `close()` command to make sure the data was properly saved in the file. (Figure 7).

```
# Save the data to a file
elif menu_choice == "3":
    try:
        file = open(FILE_NAME, "w")
        json.dump(students, file, indent=2)
        file.close()

        # Print the data to screen to tell the user what was saved to the file
        print("The following data was saved to file!")
        for student in students:
            print(f"Student {student['FirstName']} {student['LastName']} "
                  f" is enrolled in {student['CourseName']}")

    # Type Error exception
    except TypeError as e:
        print("Please check that the data is a valid JSON format\n")
        print("-- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')

    #
    except Exception as e:
        print("There was a non-specific error!")
        print("-- Technical Error Message --")
        print(e, e.__doc__, type(e), sep='\n')

    finally:
        # Check if a file object exists and is still open
        if file is not None and file.closed == False:
            file.close()
```

Figure 7. Menu Choice 3 to write the current data to the file with error management

From the starter file the fourth menu option was already complete which would end the program, breaking out of the while loop. At the end of the program a print statement tells the user that the program has ended.

Running the Program

I ran the program in the PyCharm IDE as well as the command prompt and received similar outputs, only changing with the data that was input. I ran through the program, testing to ensure that the exceptions would correctly catch errors like a mis-typed name with a number in it or a file missing from the folder. I ran through options 1 through 4 and got successful outputs for each one. (Figures 8-11).

```
---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

What would you like to do: 1
Enter the student's first name: James
Enter the student's last name: Kirk
Please enter the name of the course: Piloting 100
You have registered James Kirk for Piloting 100.
```

Figure 8. Output 1

```
---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

What would you like to do: 2
-----

The Current Data is:
Steve,Rogers,Political Science
Tony,Stark,Enginnering 100
Peter,Parker,Biology 100
Hiccup,Haddock,Dragon Training 200
James,Kirk,Piloting 100
-----
```

Figure 9. Output 2

```
---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

What would you like to do: 3
The following data was saved to file!
Student Steve Rogers is enrolled in Political Science
Student Tony Stark is enrolled in Enginnering 100
Student Peter Parker is enrolled in Biology 100
Student Hiccup Haddock is enrolled in Dragon Training 200
Student James Kirk is enrolled in Piloting 100
```

Figure 10. Output 3

```
---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

What would you like to do: 4
Program Ended
```

Figure 11. Output 4

The final data in the JSON file was correct, and was written as expected. (Figure 12).

```
[
  {
    "FirstName": "Steve",
    "LastName": "Rogers",
    "CourseName": "Political Science"
  },
  {
    "FirstName": "Tony",
    "LastName": "Stark",
    "CourseName": "Enginnering 100"
  },
  {
    "FirstName": "Peter",
    "LastName": "Parker",
    "CourseName": "Biology 100"
  },
  {
    "FirstName": "Hiccup",
    "LastName": "Haddock",
    "CourseName": "Dragon Training 200"
  },
  {
    "FirstName": "James",
    "LastName": "Kirk",
    "CourseName": "Piloting 100"
  }
]
```

Figure 12. Data in the JSON File

With the program running successfully it was time to perform a new step, uploading the files to Github.

Uploading the Files to Github

I created a Github account that I could use not only for classes but for future use. First, I uploaded my Assignment.py file and the Enrollments.json file, committing them to the repository with a comment explaining the addition. (Figure 13).

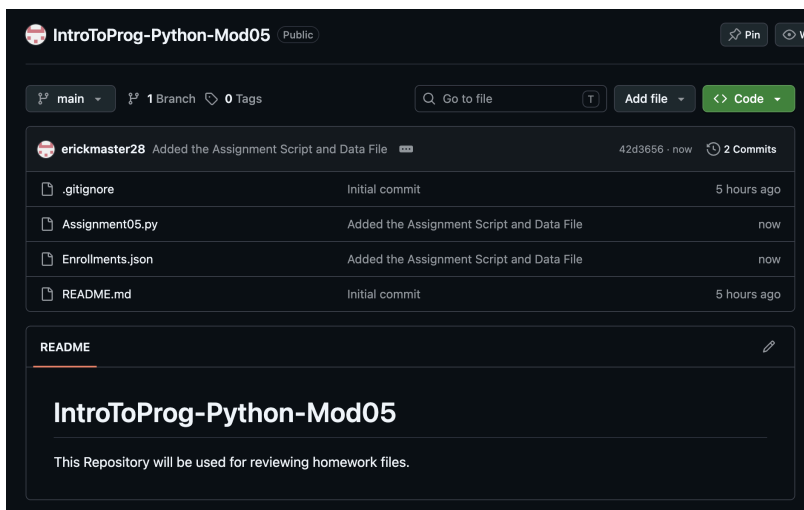


Figure 13. Github Upload

I also posted a link to the project to our discussion board for peer review, and I will add this document to the repository when it is complete. The link to the Github repository I created is here: <https://github.com/erickmaster28/IntroToProg-Python-Mod05> (external site).

Summary

In summary, this project shows the process of developing new skills and building on old skills when coding with Python. I used a program to read and write data to a JSON file based on a user's input. I also worked with error management, creating multiple catches to ensure the user input was correct and the file could be found. Lastly, I uploaded the files to Github in a repository for peer review.

Works Cited

Arya, Anubhaw. *Mod05 - Using JSON Files*. 2024.

Arya, Anubhaw. *Mod05 - Using Dictionaries with files*. 2024.

Python Software Foundation. "3.14.6 Documentation." *Python.Org*, 2019,
<https://docs.python.org/3/>.

Root, Randal. *Introduction to Programming with Python*. Self, 2026, p. Module 05.